Lab 6: Matching, Weighting, and Doubly Robust Estimation

SOCIOL 723, Social Statistics II, Thursday, October 1

Wenhao Jiang

Table of contents

1 The question

Estimand. The average difference in log personal earnings between completing a bachelor's degree and not completing one, among the full-time workers aged 25–64 in our GSS analytic

sample, 2010–2022.

> **Note**
>
> Note the deliberately modest population. The GSS is a complex sample, and doing this properly for the U.S. population would require the survey weights (`wt`) and PSU/stratum-aware inference. We work unweighted here so that the adjustment methods are visible on their own; that is a simplification, and you should say so if you write it up.

We also exclude `hours` from the adjustment set. Hours worked is plausibly a *consequence* of completing college, so conditioning on it would be the post-treatment error from Week 5; it would answer a different question.

This is an *observational* question with obvious confounding: people who complete college differ from those who do not in family background, measured ability, and much else. We are going to assume conditional ignorability given the covariates we have, do the adjustment five different ways, and then ask how much an unobserved confounder would have to matter to overturn the answer.

> **Warning**
>
> Assuming conditional ignorability here is almost certainly wrong. That is the point of the last section. Treat today's numbers as an exercise in *method*, not as an estimate of the return to college.

```
library(dplyr)
library(ggplot2)
library(MatchIt)
library(cobalt)
library(sensemakr)
library(sandwich); library(lmtest)
library(boot)

gss <- readRDS("../Data/gss_earnings.rds") |>
  mutate(D = ba, Y = lnearn)

covs <- c("pareduc", "wordsum", "female", "black", "otherace", "exper")
c(n = nrow(gss), treated = sum(gss$D), control = sum(1 - gss$D))
#>       n treated control
#>    3509    1441    2068
```

2 Where we start: the naive comparison

```
naive <- lm(Y ~ D, data = gss)
reg   <- lm(Y ~ D + pareduc + wordsum + female + black + otherace + exper +
              I(exper^2), data = gss)

c(naive = coef(naive)["D"], regression = coef(reg)["D"])
#>      naive.D regression.D
#>       0.6834       0.6295
```

Adjustment moves the estimate substantially. But regression is only one of the adjustment strategies available, and it makes assumptions we have not examined.

```
bal.tab(reg_formula <- D ~ pareduc + wordsum + female + black + otherace +
          exper, data = gss, un = TRUE, thresholds = c(m = 0.1))
#> Balance Measures
#>             Type Diff.Un     M.Threshold.Un
#> pareduc  Contin.   0.736 Not Balanced, >0.1
#> wordsum  Contin.   0.793 Not Balanced, >0.1
#> female    Binary   0.055     Balanced, <0.1
#> black     Binary  -0.053     Balanced, <0.1
#> otherace  Binary  -0.016     Balanced, <0.1
#> exper    Contin.  -0.420 Not Balanced, >0.1
#>
#> Balance tally for mean differences
#>                    count
#> Balanced, <0.1         3
#> Not Balanced, >0.1     3
#>
#> Variable with the greatest mean difference
#>  Variable Diff.Un     M.Threshold.Un
#>   wordsum   0.793 Not Balanced, >0.1
#>
#> Sample sizes
#>     Control Treated
#> All    2068    1441
```

The `Diff.Un` column is the standardized mean difference. Values well above 0.1, here `pareduc` and `wordsum` especially, mean the two groups are not comparable as they stand.

3 The propensity score

```r
ps_mod <- glm(D ~ pareduc + wordsum + female + black + otherace +
                exper + I(exper^2),
              data = gss, family = binomial)
gss$ps <- fitted(ps_mod)

summary(gss$ps)
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>  0.0027  0.2179  0.3911  0.4107  0.5900  0.9457
```

```r
ggplot(gss, aes(ps, fill = factor(D))) +
  geom_density(alpha = .45, colour = NA) +
  scale_fill_manual(values = c("0" = "navy", "1" = "firebrick"),
                    labels = c("no BA", "BA"), name = NULL) +
  labs(x = "estimated propensity score", y = "density") +
  theme_minimal(base_size = 10)
```

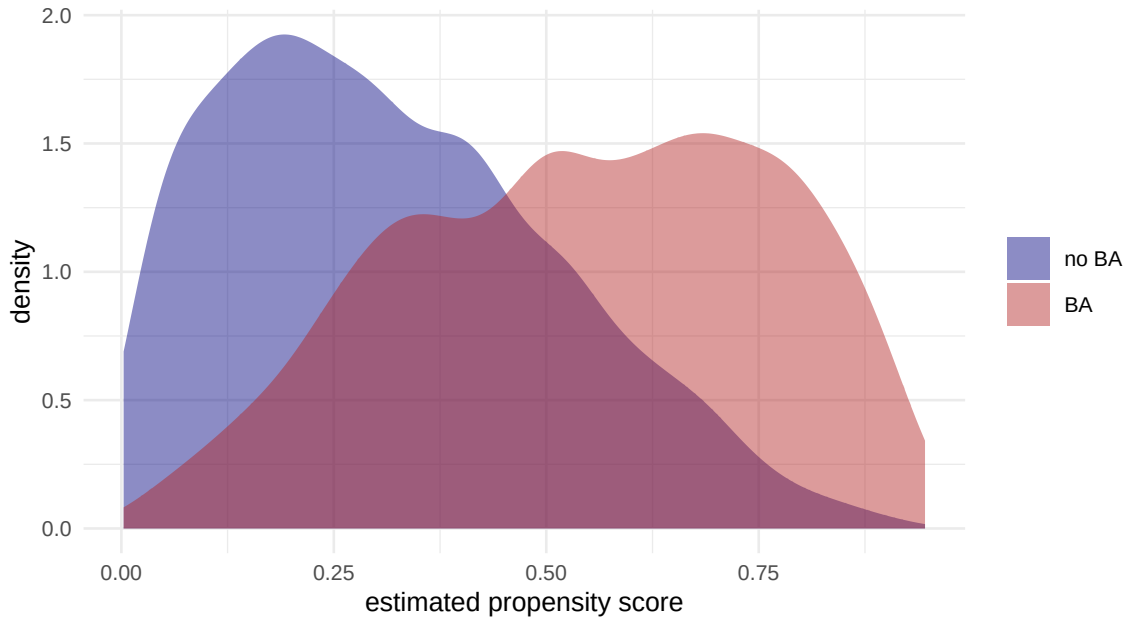

Figure 1: Distribution of the estimated propensity score by treatment status. Overlap is imperfect in both tails.

```r
## how many treated units sit where controls are scarce, and vice versa?
c(treated_above_0.9 = sum(gss$D == 1 & gss$ps > 0.9),
  controls_above_0.9 = sum(gss$D == 0 & gss$ps > 0.9),
  controls_below_0.1 = sum(gss$D == 0 & gss$ps < 0.1),
  treated_below_0.1 = sum(gss$D == 1 & gss$ps < 0.1))
#>  treated_above_0.9 controls_above_0.9 controls_below_0.1  treated_below_0.1
#>                 34                   3                 305                  28
```

> **Note**
>
> Note the trade-off from lecture: a propensity model that predicts treatment *well* is a model that separates the groups, which is exactly the situation in which comparison is hardest. Judge the model by **balance**, not by AUC.

4 Matching

4.1 Nearest-neighbour on the propensity score

```
m_nn <- matchit(D ~ pareduc + wordsum + female + black + otherace +
                  exper + I(exper^2),
                data = gss, method = "nearest", distance = "glm",
                estimand = "ATT", replace = FALSE)
m_nn
#> A `matchit` object
#>  - method: 1:1 nearest neighbor matching without replacement
#>  - distance: Propensity score             - estimated with logistic regression
#>  - number of obs.: 3509 (original), 2882 (matched)
#>  - target estimand: ATT
#>  - covariates: pareduc, wordsum, female, black, otherace, exper, I(exper^2)
```

```
love.plot(m_nn, thresholds = c(m = 0.1), binary = "std",
          colors = c("firebrick", "navy")) +
  theme_minimal(base_size = 9)
```

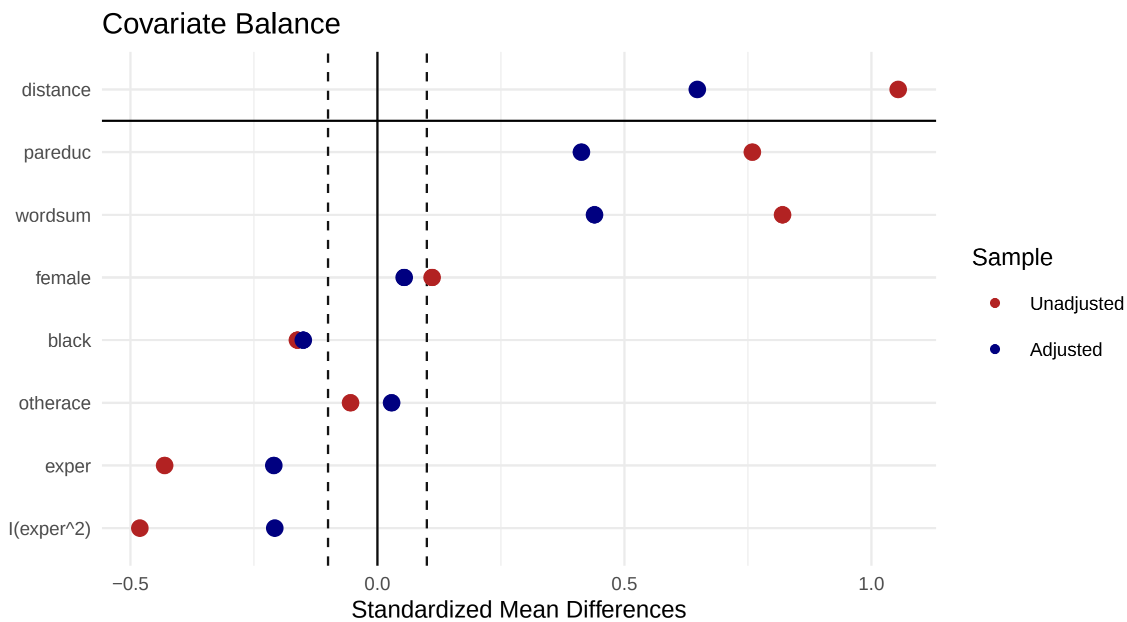

Figure 2: Covariate balance before and after nearest-neighbour matching. Everything should land inside the dashed lines.

```
md_nn <- match.data(m_nn)
fit_nn <- lm(Y ~ D + pareduc + wordsum + female + black + otherace +
               exper + I(exper^2),
             data = md_nn, weights = weights)
coeftest(fit_nn, vcov = vcovCL, cluster = ~ subclass)["D", ]
#>                                                                                 Estimate
#>  0.626002467477597357436991387658054009079933166503906250000000000000000000000000000000
#>                                                                               Std. Error
#>  0.031856715139649896539886242408101679757237434387207031250000000000000000000000000000
#>                                                                                  t value
#> 19.650565500347351388654715265147387981414794921875000000000000000000000000000000000000
#>                                                                                 Pr(>|t|)
#>  0.000000000000000000000000000000000000000000000000000000000000000000000000000000009131
```

Two things to notice. First, we still run a regression on the matched data, matching is *pre-processing*, and the regression mops up residual imbalance. Second, the standard errors are clustered on the matched pair, because the matching induces dependence.

4.2 Mahalanobis distance

Instead of collapsing the covariates to a propensity score, match on the covariates directly, scaling by their covariance so that no variable dominates through its units.

```
m_mah <- matchit(D ~ pareduc + wordsum + female + black + otherace +
                   exper,
```

```
                  data = gss, method = "nearest", distance = "mahalanobis",
                  estimand = "ATT")
summary(m_mah)$nn
#>                Control Treated
#> All (ESS)         2068    1441
#> All               2068    1441
#> Matched (ESS)     1441    1441
#> Matched           1441    1441
#> Unmatched          627       0
#> Discarded            0       0
```

4.3 A caliper

A caliper refuses matches beyond a set distance on the propensity score, measured in standard deviations of the linear predictor.

```
m_cal <- matchit(D ~ pareduc + wordsum + female + black + otherace +
                   exper + I(exper^2),
                 data = gss, method = "nearest", distance = "glm",
                 estimand = "ATT", caliper = 0.1, std.caliper = TRUE)
summary(m_cal)$nn
#>                Control Treated
#> All (ESS)         2068    1441
#> All               2068    1441
#> Matched (ESS)     1017    1017
#> Matched           1017    1017
#> Unmatched         1051     424
#> Discarded            0       0
```

Look at the **Unmatched** row for the treated. Those respondents have no comparable counterpart in the data, and dropping them silently redefines the estimand; you are now describing the subpopulation of degree-holders who have plausible non-degree counterparts.

4.4 Coarsened exact matching

```
m_cem <- matchit(D ~ pareduc + wordsum + female + black + exper,
                 data = gss, method = "cem", estimand = "ATT")
summary(m_cem)$nn
#>                Control Treated
#> All (ESS)       2068.0    1441
#> All             2068.0    1441
#> Matched (ESS)    521.1     843
#> Matched         1119.0     843
#> Unmatched        949.0     598
```

```
#> Discarded         0.0       0
```

CEM is the most transparent method: it matches exactly within coarsened bins, so balance is guaranteed by construction. The price is written plainly in the number of discarded observations.

5 Weighting

5.1 IPW by hand

```
e  <- gss$ps
D  <- gss$D
Y  <- gss$Y

## Horvitz-Thompson
ate_ht <- mean(D * Y / e) - mean((1 - D) * Y / (1 - e))

## Hajek / stabilized -- divide by the sum of the weights
w1 <- D / e
w0 <- (1 - D) / (1 - e)
ate_sipw <- sum(w1 * Y) / sum(w1) - sum(w0 * Y) / sum(w0)

c(horvitz_thompson = ate_ht, stabilized = ate_sipw)
#> horvitz_thompson       stabilized
#>           1.5371           0.6023
```

```
w <- ifelse(D == 1, 1 / e, 1 / (1 - e))
ess <- sum(w)^2 / sum(w^2)
c(max_weight = max(w), ess = ess, n = length(w), ess_share = ess / length(w))
#> max_weight        ess          n  ess_share
#>   105.4685  1241.7073  3509.0000     0.3539
```

The **effective sample size** is what your standard errors should reflect. If a handful of observations carry most of the weight, you have far less information than n suggests.

5.2 ATT and overlap weights

```
att_w  <- ifelse(D == 1, 1, e / (1 - e))          # ATT weights
ato_w  <- ifelse(D == 1, 1 - e, e)                # overlap weights

wmean_diff <- function(wt) {
  sum(wt * Y * D) / sum(wt * D) - sum(wt * Y * (1 - D)) / sum(wt * (1 - D))
}
c(ATE_stabilized = ate_sipw,
  ATT = wmean_diff(att_w),
```

```
  ATO_overlap = wmean_diff(ato_w))
#> ATE_stabilized            ATT    ATO_overlap
#>         0.6023         0.6310         0.6371
```

Three different numbers, because they are three different **estimands**, not three attempts at one. The overlap-weighted estimate describes the subpopulation where comparison is most informative, and its weights are bounded by construction.

5.3 Does weighting actually balance the covariates?

```
bal.tab(D ~ pareduc + wordsum + female + black + otherace + exper,
        data = gss, weights = ifelse(D == 1, 1 / e, 1 / (1 - e)),
        method = "weighting", un = TRUE, thresholds = c(m = 0.1))
#> Balance Measures
#>             Type Diff.Un Diff.Adj        M.Threshold
#> pareduc  Contin.   0.736   -0.218 Not Balanced, >0.1
#> wordsum  Contin.   0.793   -0.077     Balanced, <0.1
#> female    Binary   0.055   -0.027     Balanced, <0.1
#> black     Binary  -0.053    0.026     Balanced, <0.1
#> otherace  Binary  -0.016    0.021     Balanced, <0.1
#> exper    Contin.  -0.420    0.006     Balanced, <0.1
#>
#> Balance tally for mean differences
#>                     count
#> Balanced, <0.1          5
#> Not Balanced, >0.1      1
#>
#> Variable with the greatest mean difference
#>  Variable Diff.Adj        M.Threshold
#>   pareduc   -0.218 Not Balanced, >0.1
#>
#> Effective sample sizes
#>            Control Treated
#> Unadjusted    2068  1441.
#> Adjusted      1623   410.4
```

6 G-computation and AIPW

```
## fit separate outcome models in each arm, then impute both potential outcomes
out_form <- Y ~ pareduc + wordsum + female + black + otherace + exper +
  I(exper^2)
m1 <- lm(out_form, data = gss[D == 1, ])
```

```r
m0 <- lm(out_form, data = gss[D == 0, ])

mu1 <- predict(m1, newdata = gss)
mu0 <- predict(m0, newdata = gss)

ate_gcomp <- mean(mu1 - mu0)
ate_gcomp
#> [1] 0.6318
```

```r
aipw_terms <- (mu1 - mu0) +
  D * (Y - mu1) / e -
  (1 - D) * (Y - mu0) / (1 - e)

ate_aipw <- mean(aipw_terms)

## the influence-function standard error comes free
se_aipw <- sd(aipw_terms) / sqrt(length(aipw_terms))

c(estimate = ate_aipw, se = se_aipw,
  lower = ate_aipw - 1.96 * se_aipw, upper = ate_aipw + 1.96 * se_aipw)
#> estimate       se    lower    upper
#>  0.60512  0.03981  0.52709  0.68315
```

> **Tip**
>
> The AIPW estimator is an average of n i.i.d. terms, so its standard error is just the standard deviation of those terms over $\sqrt{n}$. That is not a coincidence, `aipw_terms` is the *efficient influence function*, and this is the pattern that Week 13 generalizes.

6.1 Everything in one place

```r
results <- tibble::tibble(
  method = c("Naive difference", "OLS adjustment", "NN matching (ATT)",
             "IPW stabilized (ATE)", "ATT weights", "Overlap weights (ATO)",
             "G-computation (ATE)", "AIPW (ATE)"),
  estimate = c(coef(naive)["D"], coef(reg)["D"], coef(fit_nn)["D"],
               ate_sipw, wmean_diff(att_w), wmean_diff(ato_w),
               ate_gcomp, ate_aipw)
)
knitr::kable(results, digits = 4)
```

method	estimate
Naive difference	0.6834
OLS adjustment	0.6295
NN matching (ATT)	0.6260
IPW stabilized (ATE)	0.6023
ATT weights	0.6310
Overlap weights (ATO)	0.6371
G-computation (ATE)	0.6318
AIPW (ATE)	0.6051

If these had disagreed sharply, that itself would be the finding: it would tell us the answer is coming from modelling choices rather than from comparison.

6.2 Bootstrapping the whole procedure

Standard errors must account for the fact that the propensity score was *estimated*. The safest route is to bootstrap the entire pipeline.

```
aipw_fun <- function(data, idx) {
  d <- data[idx, ]
  psm <- glm(D ~ pareduc + wordsum + female + black + otherace +
               exper + I(exper^2), data = d, family = binomial)
  ee  <- fitted(psm)
  f1 <- lm(out_form, data = d[d$D == 1, ]); f0 <- lm(out_form, data = d[d$D == 0, ])
  m1p <- predict(f1, newdata = d); m0p <- predict(f0, newdata = d)
  mean((m1p - m0p) + d$D * (d$Y - m1p) / ee -
         (1 - d$D) * (d$Y - m0p) / (1 - ee))
}
set.seed(723)
bo <- boot(gss, aipw_fun, R = 500)
c(estimate = bo$t0, boot_se = sd(bo$t), analytic_se = se_aipw)
#>    estimate     boot_se analytic_se
#>     0.60512     0.03579     0.03981
```

The two standard errors agree closely, which is reassuring: the influence-function formula is doing its job.

7 Sensitivity analysis

All of the above assumed we measured every confounder. We did not. So ask instead: **how strong would an unobserved confounder have to be?**

```r
sens <- sensemakr(model = reg, treatment = "D",
                  benchmark_covariates = "wordsum",
                  kd = 1:3)
sens
#> Sensitivity Analysis to Unobserved Confounding
#>
#> Model Formula: Y ~ D + pareduc + wordsum + female + black + otherace + exper +
#>     I(exper^2)
#>
#> Null hypothesis: q = 1 and reduce = TRUE
#>
#> Unadjusted Estimates of ' D ':
#>   Coef. estimate: 0.629
#>   Standard Error: 0.032
#>   t-value: 19.98
#>
#> Sensitivity Statistics:
#>   Partial R2 of treatment with outcome: 0.102
#>   Robustness Value, q = 1 : 0.285
#>   Robustness Value, q = 1 alpha = 0.05 : 0.262
#>
#> For more information, check summary.
```

```r
plot(sens)
```

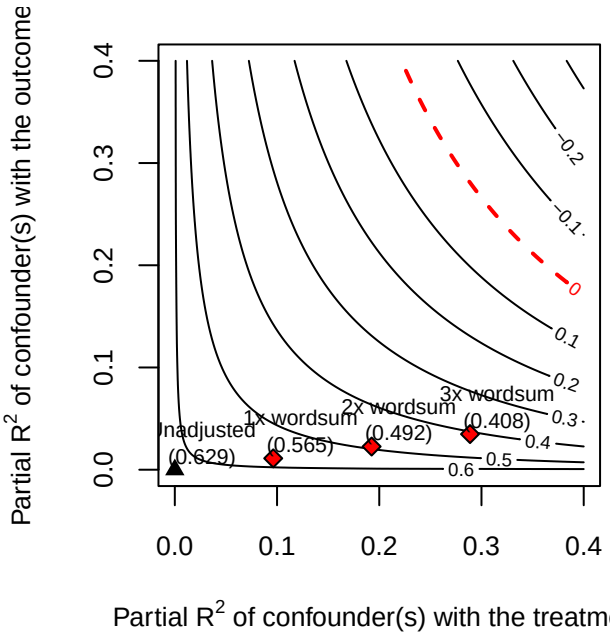

Figure 3: Contours of the adjusted estimate as a function of how much residual variation in treatment and outcome an unobserved confounder explains. Benchmarks show multiples of the vocabulary score.

Read the **robustness value**: it is the share of residual variation in both the treatment and the outcome that a confounder would need to explain to reduce the estimate to zero. Compare it to the benchmark: if a confounder three times as strong as measured verbal ability would be needed, the finding is reasonably robust; if half of one would do it, it is not.

8 Exercises

1. **Estimand discipline.** Redo the analysis targeting the **ATT** throughout (matching, ATT weights, and an ATT version of AIPW). Do the answers move? Explain why the ATE and ATT differ here in terms of who is on the margin of completing college.

2. **Trimming.** Restrict to $0.1 < \hat{e} < 0.9$ and recompute IPW and AIPW. How much does the estimate move, and how many observations did you drop? Write one sentence stating what population your new estimate describes.

3. **Deliberate misspecification, on simulated data.** Double robustness cannot be "confirmed" against observational data, because the causal truth is unknown. Simulate a world with a known ATE, then fit (a) a correct outcome model with a badly wrong propensity model and (b) the reverse. Confirm that AIPW stays close to the truth in both cases while IPW and g-computation each fail in one. This is double robustness in action.

4. **A bad control.** Add `prestige` (occupational prestige) to the covariate set. It is plausibly a *consequence* of holding a degree. Report how the estimate changes and explain, using the Week 5 vocabulary, why the new number answers a different question.

5. **Balance versus fit.** Compute the AUC of the propensity model. Then add interactions until balance improves. Does the AUC go up or down? Discuss what this means for the "propensity score paradox" from lecture.

6. **Effective sample size.** Plot the IPW weights against $\hat{e}$. Identify the ten most heavily weighted observations. Refit without them, how much does the estimate move relative to its standard error?

9 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] boot_1.3-30     lmtest_0.9-40   zoo_1.8-12      sandwich_3.1-1
#> [5] sensemakr_0.1.6 cobalt_4.6.1    MatchIt_4.6.0   ggplot2_4.0.0
#> [9] dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#>  [1] gtable_0.3.6       jsonlite_1.8.9     compiler_4.4.1     crayon_1.5.3
#>  [5] tinytex_0.53       tidyselect_1.2.1   Rcpp_1.1.1         scales_1.4.0
#>  [9] yaml_2.3.10        fastmap_1.2.0      lattice_0.22-7     R6_2.5.1
#> [13] labeling_0.4.3     generics_0.1.3     knitr_1.48         backports_1.5.0
#> [17] tibble_3.2.1       chk_0.10.0         pillar_1.9.0       RColorBrewer_1.1-3
#> [21] rlang_1.3.0        utf8_1.2.4         xfun_0.47          S7_0.2.0
#> [25] cli_3.6.5          withr_3.0.1        magrittr_2.0.3     digest_0.6.37
#> [29] grid_4.4.1         lifecycle_1.0.4    vctrs_0.6.5        evaluate_1.0.0
```

```
#> [33] glue_1.7.0          farver_2.1.2       fansi_1.0.6        rmarkdown_2.28
#> [37] tools_4.4.1         pkgconfig_2.0.3    htmltools_0.5.8.1
```